



Presentación Grupal ISWI Investigación y Aplicación de Conceptos

Objetivo

Desarrollar habilidades de investigación, análisis, aplicación práctica y comunicación efectiva. A diferencia de una exposición teórica, cada equipo debe resolver un problema real construyendo una porción funcional de software y demostrando, sobre ese caso, cómo se aplican de forma integrada el ciclo de vida del software, la calidad y la seguridad. Se busca desarrollar las siguientes competencias en su etapa de transición:

- ❖ **Liderazgo solidario y transformador** — trabajo colaborativo.
- ❖ **Gestión del conocimiento técnico-científico** — integración de conocimiento — Investigación.
- ❖ **Diseño sistémico de procesos de ingeniería** — diseño integrado y ejecución de procesos de ingeniería.

1. Contexto de la presentación

No se trata de “exponer un tema”, sino de **resolver un problema real y demostrarlo funcionando**. Cada equipo:

- ❖ Elige un problema o producto real (**dominio libre**): una necesidad concreta que el equipo entienda y pueda acotar.
- ❖ Aplica su **tema asignado como lente de ingeniería obligatoria** para diseñar y construir la solución.
- ❖ Demostrar el acoplamiento del **Ciclo de Vida del Software**, considerando: requisitos → arquitectura → configuración → calidad → pruebas → seguridad → despliegue → operación.

2. Temas asignables

Cada equipo recibirá uno de los siguientes temas (el orden de asignación se sorteará en clase):

- ❖ **DevSecOps**
- ❖ **Microservicios vs. Monolito** (incluye la evolución SOA a microservicios).
- ❖ **EDA — Arquitectura Orientada a Eventos**
- ❖ **Aplicaciones potenciadas por IA / agentes (RAG)**
- ❖ **Observabilidad / SRE** (Site Reliability Engineering).
- ❖ **Cloud Native** — Contenedores + Serverless.
- ❖ **Blockchain / Distributed Ledger**

3. Base común obligatoria (aplica a todos los equipos)

Sin importar el tema o el dominio elegido, **todo equipo debe entregar y demostrar lo siguiente**.

3.1 Prototipo

El prototipo **no es el producto completo**: es **una sola funcionalidad de punta a punta** que ejemplifique el tema asignado, que se pueda ejecutar y mostrar en vivo. Preferir lo profundo y funcional sobre lo amplio e incompleto. Ver la Sección 4 para qué cuenta como prototipo en cada tema.

3.2 Entregables obligatorios



N°	Entregable	Qué incluye (evidencia)
1	Mini-SRS	3 a 5 requisitos + 1 o 2 casos de uso o historias de usuario que acoten el alcance.
2	Diagrama de arquitectura (C4, nivel contenedor)	Diagrama + justificación de los trade-offs de la decisión arquitectónica (por qué esta arquitectura y no otra).
3	Plan de calidad	2 o 3 atributos de calidad ISO/IEC 25010 priorizados para el caso, y cómo se prueban.
4	Análisis de seguridad	Tríada CIA aplicada al caso + 2 o 3 riesgos OWASP pertinentes y su mitigación.
5	Prototipo + repositorio + Kanban	Demo ejecutable, en un repositorio con historial, y el Kanban con las tareas de cada integrante.
6	Demo en vivo	Bloque cronometrado y calificado dentro de la presentación (ver 3.3).

3.3 Estructura y tiempos de la presentación (20–25 min) – Estructura sugerida

Segmento	Tiempo
Problema y contexto (qué necesidad se resuelve)	2 min
Arquitectura y decisiones de diseño (C4 + trade-offs)	5 min
Aplicación del ciclo de vida: calidad y seguridad sobre el caso	6 min
Demo en vivo del Prototipo (obligatoria)	7–8 min
Lecciones aprendidas y cierre	2 min
Preguntas al equipo	2 min

3.4 Hitos evaluados (compuertas de avance)

- ❖ **Hito 1** — Problema elegido + Mini-SRS + Diagrama de arquitectura.
- ❖ **Hito 2** — Ensayo (dry-run) de la demo del prototipo funcionando.
- ❖ **Entrega final + presentación.**

4. ¿Qué cuenta como prototipo? (por tema)

Referencia para acotar el alcance. En todos los casos el prototipo debe ejecutarse en vivo.

Tema	Prototipo mínimo y qué debe mostrar la demo	No cuenta como prototipo
DevSecOps	Pipeline CI/CD real sobre un repositorio: un commit dispara build → pruebas → escaneo de seguridad (SAST + análisis de dependencias/SCA; DAST opcional) → despliegue.	Hablar de DevSecOps sin un pipeline ejecutándose.
Microservicios vs. Monolito	Una funcionalidad descompuesta en 2 o 3 servicios independientes con un API Gateway (preferiblemente), y la justificación frente a un monolito modular.	Un solo servicio llamado “microservicio”, o solo diagramas sin sistema.



Tema	Prototipo mínimo y qué debe mostrar la demo	No cuenta como prototipo
EDA (Event-Driven)	Un broker de mensajes con un productor y al menos dos consumidores que procesan un evento de forma asíncrona.	Llamadas síncronas REST disfrazadas de "eventos".
Apps con IA / agentes (RAG)	Una función con recuperación aumentada (RAG): ingesta de documentos → vector store → consulta que recupera contexto y responde citando fuentes; o un agente que use ≥ 1 herramienta.	Un chatbot que solo reenvía prompts, sin recuperación ni control de alucinaciones.
Observabilidad / SRE	Un servicio instrumentado con logs estructurados + métricas + trazas (OpenTelemetry) y un dashboard; definir 1 SLO con su alerta.	Un dashboard estático, sin instrumentación real ni SLO.
Cloud Native (Containers + Serverless)	Contenerizar el servicio (Dockerfile) y desplegarlo orquestado (Kubernetes/compose) o como función serverless con un disparador; idealmente con IaC.	Correr la app localmente sin contenedor ni despliegue.
Blockchain / DLT	Un smart contract o registro sobre una red permissionada (Hyperledger Fabric) o testnet, que resuelva un caso concreto de trazabilidad/provenance o verificación (p. ej. cadena de custodia de un lote). Demo: escribir una transacción y que otra parte la lea y verifique su inmutabilidad (mostrar el hash/prueba).	Un "token de juguete" sin caso de negocio, o diapositivas sobre blockchain sin un ledger corriendo.

5. Instrucciones para el equipo

- ❖ El trabajo se realiza en los equipos previamente formados en clase.
- ❖ Elijan un problema real y acótenlo.
- ❖ Investiguen el tema asignado para tomar decisiones de diseño fundamentadas (no para llenar diapositivas de teoría).
- ❖ Construyan el prototipo y documenten los 6 entregables de la Sección 3.2.
- ❖ Lleven las tareas de cada integrante en un Kanban, que deben adjuntar a la documentación y presentación.
- ❖ Habrá espacios durante las clases para resolución de dudas; se recomienda trabajar desde la fecha de entrega de este enunciado.

6. Criterios de evaluación

- ❖ Claridad y profundidad de la investigación aplicada a las decisiones de diseño.
- ❖ Relevancia del problema elegido y pertinencia de la solución.
- ❖ Dominio del tema durante la exposición y en las preguntas individuales.
- ❖ Calidad de la arquitectura y de los entregables de ciclo de vida (calidad y seguridad).
- ❖ Efectividad de la demostración en vivo del prototipo.



7. Rúbrica de calificación

Trabajo colaborativo	20% – 2 pts.
Integración de conocimiento	20% – 2 pts.
Investigación	20% – 2 pts.
Diseño integrado (arquitectura + prototipo)	20% – 2 pts.
Ejecución de procesos de ingeniería (ciclo de vida + demo)	20% – 2 pts.
Total	100% – 10 pts.

Rúbrica de calificación por competencias

Criterio	Alcanzado (20)	En proceso alto (15)	En proceso bajo (08)	Puede mejorar (04)
Trabajo colaborativo	Coordinación y reparto equitativo evidenciados en el Kanban; todos participan y pueden defender cualquier parte.	Trabajan bien en conjunto, con desigualdades menores en participación o coordinación.	Dependencia de pocos miembros; comunicación dispersa; Kanban incompleto.	Sin evidencia de trabajo colaborativo; proyecto hecho por pocos o con conflictos.
Integración de conocimiento	Conecta de forma coherente requisitos, arquitectura, calidad y seguridad sobre el caso; el tema se ve aplicado, no expuesto.	Integra los conceptos con menor profundidad o alguna conexión floja.	Integra conceptos de forma superficial o solo en la exposición, no en el prototipo.	Conceptos sueltos o mayormente teóricos; poca relación con el caso.
Investigación	Fuentes confiables y actuales que sustentan las decisiones de diseño; citadas correctamente; uso de IA declarado y comprendido.	Fuentes pertinentes, con menor variedad o profundidad.	Fuentes limitadas o poco relevantes; sustento débil de las decisiones.	Sin evidencia de investigación o información incorrecta; uso de IA no declarado ni comprendido.
Diseño integrado	Arquitectura (C4) completa, coherente y justificada por trade-offs; prototipo (rebanada vertical) funcional y alineado con los requisitos.	Diseño adecuado con elementos faltantes o poco detallados; prototipo funcional parcial.	Diseño básico o incompleto; prototipo mínimo o que apenas corre.	Diseño sin relación con el requisito o sin prototipo funcional.
Ejecución de procesos de ingeniería	Aplica el ciclo de vida (requisitos, diseño, calidad, seguridad, despliegue) de forma estructurada; la demo en vivo evidencia el funcionamiento.	Aplica los procesos con algunas omisiones o simplificaciones; la demo funciona en su mayoría.	Aplica los procesos de forma parcial o con errores; demo limitada o con fallos.	No aplica los procesos o lo hace de forma desorganizada; sin demo funcional.



8. Fechas importantes

La fecha de presentación es en horario de clase y el orden se sorteará entre los grupos.

- ❖ Entrega del enunciado: 07/09/2026
- ❖ Hito 1 (problema + Mini-SRS + arquitectura): 12/10/2026
- ❖ Hito 2 (dry-run de la demo): 21/10/2026
- ❖ Entrega final en el portal + presentaciones: 02/11/2026 y 04/11/2026

9. Consideraciones

- ❖ Este proyecto se trabaja de forma grupal.
- ❖ Durante la presentación se harán preguntas del tema a cada integrante para validar los conceptos.
- ❖ Las referencias a fuentes o material de terceros deben estar documentadas, referenciadas y justificado su uso.
- ❖ **Uso de IA:** se permite y se espera como herramienta de trabajo, pero debe declararse, comprenderse y poder defenderse. Todo lo que el equipo presente debe poder explicarse sin la IA.
- ❖ El material se entrega en el espacio habilitado del portal previo a la presentación.
- **COPIA PARCIAL O TOTAL DEL PROYECTO TENDRÁ UNA NOTA DE 0 PUNTOS, Y SE NOTIFICARÁ A LA COORDINACIÓN DEL ÁREA PARA QUE SE APLIQUEN LAS SANCIONES CORRESPONDIENTES A LOS ALUMNOS DEL EQUIPO DE TRABAJO.**